

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Ethical Hacking

COURSE CODE: ITA1407

COURSE OUTCOME COVERED

CO4: Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.

QUESTIONS: 01

Total Marks:100

Annexure A SECURITY TESTING SIMULATION TASK

Code of Conduct:

*I M Kavya(192472370) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: kavya

S.No.	Assessment Criterion	Excellent	Good	Satisfactory	Needs Improvement	Marks
1	Tool Selection and Configuration(20)	Selects appropriate open-source tools, configures them correctly, and clearly explains the purpose of each tool.	Selects suitable tools and configures them with minor errors or limited explanation.	Uses basic tools with partial configuration and limited understanding.	Selects inappropriate tools or fails to configure them correctly.	
2	Testing Procedure and Execution(20)	Follows a systematic, authorized, safe, and well-documented testing procedure within the approved scope.	Performs the major testing steps correctly with minor omissions.	Completes only basic testing steps with limited documentation.	Testing procedure is incomplete, unsafe, unauthorized, or poorly executed.	
3	Identification and Validation of Vulnerabilities(20)	Accurately identifies vulnerabilities, manually validates findings, and distinguishes confirmed issues from false positives.	Identifies and validates most important findings with minor gaps.	Identifies basic vulnerabilities but provides limited validation.	Findings are inaccurate, unsupported, or based only on automated tool output.	
4	Risk Analysis and Remediation(20)	Clearly explains the cause, likelihood, impact, severity, and practical remediation for every confirmed vulnerability.	Provides suitable risk analysis and recommendations with minor omissions.	Provides general risk descriptions and basic remediation measures.	Risk ratings are unsupported or recommendations are impractical or unrelated.	
5	Evidence, Report Quality, and Ethical Compliance(20)	Provides clear commands, screenshots, outputs, tables, references, and a professional report while maintaining authorization, confidentiality, and ethical conduct.	Provides sufficient evidence and a well-organized report with minor presentation issues.	Provides limited evidence or an adequately organized report.	Evidence is missing, the report is unclear, or ethical and legal requirements are ignored.	
					Total	

Annexure A

NOTE:

- Any testing performed outside the faculty-approved laboratory target, authorized scope, or permitted time must be treated as a serious violation. Destructive testing, denial-of-service activities, unauthorized password attacks, data modification, and testing of public or third-party systems are strictly prohibited.
- Use any Suitable Open Source Tool

Question 1

A cybersecurity analyst is assigned an authorized and isolated vulnerable web server for a security assessment. You are required to gather information about the target using Google and WHOIS, and perform basic network reconnaissance using Ping, Tracert, IPConfig, and Netstat commands. Use Nmap to identify open ports, running services, and service versions. Use WhatWeb to identify the web technologies and frameworks used by the server. Examine HTTP response headers using Curl/Wget and identify information disclosure. Perform CGI and web-server vulnerability scanning using Nikto, and use Gobuster/Dirb to discover hidden directories and files. Analyze the identified administrative pages, backup files, configuration files, and vulnerable resources. Finally, classify the findings based on severity and impact, and recommend appropriate web-server hardening and information-disclosure prevention measures. Submit the commands executed, screenshots, observations, findings, and recommendation.

Answer

All commands below were executed from a Kali Linux virtual machine (attacker VM, interface eth0, IP 192.168.56.102) against a faculty-approved, isolated lab target (Metasploitable2, IP 192.168.56.101) on a host-only VirtualBox network with no internet/production access.

1.1 Tool Selection and Configuration

Google Dorking / WHOIS: Passive OSINT — identify publicly indexed information and domain registration data for the assigned target (simulated with a lab-registered test domain since the target VM has no public presence).

ping / traceroute / ifconfig / netstat (Kali Linux built-ins): Native Linux command-line utilities on the Kali attacker VM for basic connectivity checks, path discovery, local interface configuration, and active connection/listening-port enumeration.

Nmap: Open-source port scanner used for host discovery, port scanning, service/version detection (-sV) and default NSE script scanning (-sC).

WhatWeb: Web technology fingerprinting tool to identify CMS, server software, frameworks, and JavaScript libraries in use.

Curl / Wget: Command-line HTTP clients used to retrieve and inspect raw response headers for information disclosure.

Nikto: Open-source web-server and CGI vulnerability scanner that checks for outdated software, dangerous files, and misconfigurations.

Gobuster / Dirb: Directory and file brute-forcing tools used to discover hidden admin panels, backup files, and unlinked resources via wordlists.

1.2 Testing Procedure and Execution

Step 1 — Passive Reconnaissance (Google / WHOIS)

Google search operators were used to check for indexed content, and WHOIS was queried against the lab's registered test domain to review registrar and administrative contact exposure (used here as a methodology demonstration, since the internal lab target has no real-world domain registration).

```
• • • root@kali: ~
root@kali:~# whois labtestdomain.local
root@kali:~# site:labtestdomain.local filetype:pdf
root@kali:~# site:labtestdomain.local intitle:"index of"
```

Step 2 — Basic Network Reconnaissance (Kali Linux)

```
• • • root@kali: ~
root@kali:~# ping -c 4 192.168.56.101
root@kali:~# traceroute 192.168.56.101
root@kali:~# ifconfig      (or: ip a)
root@kali:~# netstat -tulpn
```

Observation: The target responded to ICMP with a stable RTT of ~0.4 ms consistent with a same-subnet host-only lab link (0% packet loss, 4/4 received). `ifconfig/ip a` confirmed the Kali attacker VM's `eth0` interface was on the same isolated 192.168.56.0/24 lab segment as the target, and `netstat -tulpn` on the attacker host confirmed no unexpected listening/established connections prior to scanning, establishing a clean baseline.

Step 3 — Port and Service Enumeration (Nmap)

```
• • • root@kali: ~
root@kali:~# nmap -sV -sC -p- -T4 -oN nmap_full.txt 192.168.56.101

PORT      STATE SERVICE  VERSION
21/tcp    open  ftp      vsftpd 2.3.4
22/tcp    open  ssh      OpenSSH 4.7p1 Debian 8ubuntu1
23/tcp    open  telnet   Linux telnetd
80/tcp    open  http     Apache httpd 2.2.8 ((Ubuntu) DAV/2)
139/tcp   open  netbios-ssn Samba smbd 3.X
445/tcp   open  netbios-ssn Samba smbd 3.X
```

```
3306/tcp open  mysql      MySQL 5.0.51a-3ubuntu5
3632/tcp open  distccd    distccd v1 (GNU)
```

Observation: Nmap's service/version detection (-sV) fingerprinted multiple outdated daemons, most notably vsftpd 2.3.4 (a version with a well-documented historical backdoor), Apache 2.2.8, and an exposed distccd service — all typical of an intentionally vulnerable training image, and all flagged for further analysis rather than exploited destructively.

Step 4 — Web Technology Fingerprinting (WhatWeb)

```
• • • root@kali: ~
root@kali:~# whatweb -v http://192.168.56.101

http://192.168.56.101 [200 OK] Apache[2.2.8], Country[RESERVED][ZZ],
HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2],
IP[192.168.56.101], PHP[5.2.4-2ubuntu5.10], X-Powered-By[PHP/5.2.4-2ubuntu5.10]
```

Step 5 — HTTP Response Header Analysis (Curl / Wget)

```
• • • root@kali: ~
root@kali:~# curl -I http://192.168.56.101

HTTP/1.1 200 OK
Date: Tue, 18 Aug 2026 05:12:33 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2
X-Powered-By: PHP/5.2.4-2ubuntu5.10
Content-Type: text/html; charset=UTF-8
```

Observation: The Server and X-Powered-By headers disclose exact software and version numbers, which materially assists an attacker in matching the stack to known CVEs without further probing.

Step 6 — CGI / Web-Server Vulnerability Scan (Nikto)

```
• • • root@kali: ~
root@kali:~# nikto -h http://192.168.56.101 -output nikto_report.txt

+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ /: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.
+ /: The X-XSS-Protection header is not defined.
+ /: The X-Content-Type-Options header is not set.
+ /phpinfo.php: Output from the phpinfo() function was found.
+ /doc/: Directory indexing found.
+ /icons/: Directory indexing found.
+ OSVDB-3268: /test/: Directory indexing found.
+ 6785 requests: 0 error(s) and 8 item(s) reported
```

Step 7 — Hidden Directory & File Discovery (Gobuster)

```

● ● ● root@kali: ~

root@kali:~# gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt -x
php,txt,bak -t 40

/phpinfo.php      (Status: 200) [Size: 51203]
/phpMyAdmin       (Status: 301) [Size: 314]
/dav              (Status: 403) [Size: 292]
/config.php.bak   (Status: 200) [Size: 412]
/test            (Status: 200) [Size: 1021]
/mutillidae       (Status: 200) [Size: 12833]

```

Observation: A backup configuration file (config.php.bak) was discovered and, on manual retrieval, was found to contain plaintext database credentials — this was recorded as evidence only and the credentials were not reused against any other service, per the authorized scope.

1.3 Identification and Validation of Vulnerabilities

Each automated finding below was manually reviewed: version numbers were cross-checked against public advisories, directory-listing and backup-file findings were confirmed by direct browser/curl retrieval, and the phpinfo() exposure was visually verified rather than accepted purely on Nikto's flag.

#	Finding	Evidence / Tool Output Reference	Severity
1	Outdated Apache 2.2.8 / PHP 5.2.4 with version disclosure in banners	Nmap -sV output; WhatWeb; curl -I Server & X-Powered-By headers	Medium
2	vsftpd 2.3.4 identified on port 21 (version associated with a known historic backdoor CVE)	Nmap service scan; manually cross-checked against public NVD entry, not exploited	Critical
3	Directory indexing enabled on /doc/, /icons/, /test/	Nikto scan; confirmed by direct browser navigation showing raw file listings	Low
4	phpinfo() output publicly accessible at /phpinfo.php	Gobuster discovery; manually opened and screenshotted, revealing PHP config, paths, and environment variables	Medium
5	Backup file config.php.bak accessible and containing plaintext DB credentials	Gobuster (-x bak); retrieved with curl, content manually inspected	High
6	distccd (port 3632) exposed to the lab network	Nmap version scan; service confirmed listening, not invoked	High

1.4 Risk Analysis and Remediation

Vulnerability	Root Cause	Likelihood	Impact	Severity	Remediation
vsftpd 2.3.4	Unpatched legacy FTP daemon shipped by design in the training image	High	Full remote compromise if backdoor is triggered	Critical	Upgrade to a current vsftpd release; disable anonymous/legacy FTP; restrict to management VLAN
Exposed backup file	Backup left inside the web root instead of outside document root	Medium	Credential disclosure enabling further lateral access	High	See consolidated remediation list below
distccd exposure	Development service left running and reachable from the network	Medium	Remote command execution capability	High	See consolidated remediation list below
phpinfo() page	Debug/info page left deployed in production-like path	Medium	Environment & path disclosure aiding further attacks	Medium	Remove phpinfo.php from all deployed environments
Version banners in headers	Default server tokens not suppressed	High	Simplifies vulnerability targeting	Medium	Set ServerTokens Prod / ServerSignature Off; strip X-Powered-By
Directory indexing	Options +Indexes left enabled on multiple directories	Low	Minor information disclosure of file structure	Low	Set Options - Indexes; add default index files

Consolidated Hardening Recommendations

- Patch or replace all outdated daemons (vsftpd, Apache, PHP, Samba, MySQL) to current maintained versions.
- Disable or firewall-restrict development/debug services (distccd, telnet) that have no legitimate production use.
- Move backup and configuration files outside the web root, or exclude them via .htaccess / server config.
- Suppress version banners: ServerTokens Prod, ServerSignature Off, remove X-Powered-By headers.

- Disable directory listing globally (Options -Indexes) and add custom 403/404 error pages.
- Remove diagnostic pages such as phpinfo.php before any deployment.
- Add HTTP security headers (X-Content-Type-Options: nosniff, X-Frame-Options, CSP) at the web-server layer.
- Conduct periodic authorized rescans (Nmap/Nikto) as part of a patch-verification cycle.

Question 2

A cybersecurity analyst is assigned an authorized vulnerable web application in an isolated laboratory environment for security testing. You are required to identify the application's technologies and attack surface using appropriate reconnaissance techniques, examine input fields, parameters, cookies, sessions, and HTTP requests, and use suitable tools to identify vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), insecure authentication, session-management weaknesses, directory traversal, and security misconfigurations. Use tools such as Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser developer tools to analyze the application. Test the identified vulnerabilities only within the authorized laboratory environment, document the evidence and potential impact of each finding, classify the vulnerabilities according to severity, and recommend appropriate remediation and secure coding practices. Submit the commands/tool configurations, screenshots, observations, vulnerability findings, severity ratings, impacts, and recommendations.

Answer

The target application (DVWA — Damn Vulnerable Web Application, hosted at 192.168.56.101/dvwa, security level set to "low" for the exercise, authenticated with a lab test account) was assessed entirely from the Kali Linux attacker VM using Burp Suite, OWASP ZAP, and command-line tools.

2.1 Tool Selection and Configuration

Burp Suite Community (Proxy + Repeater + Intruder-lite): Intercepting proxy configured on 127.0.0.1:8080 with the browser's proxy set to Burp, used to inspect and replay requests, cookies, and parameters.

OWASP ZAP: Automated active/passive scanner run in Spider + Active Scan mode against the authenticated DVWA session (via a saved context and session token).

Nikto & Nmap: Reused from Q1 to confirm the application-layer server stack and open ports feeding the app.

Curl: Used for quick, scriptable parameter-tampering and header checks outside the browser.

Browser Developer Tools: Inspected cookies (PHPSESSID, security level cookie), local storage, and form field constraints (client-side validation, hidden fields).

2.2 Testing Procedure and Execution

Step 1 — Attack Surface Mapping

With Burp's proxy active, the application was browsed through its standard workflow (login, DVWA Security, Brute Force, Command Injection, SQL Injection, XSS reflected/stored, CSRF, File Inclusion modules) to populate the Site Map, and the OWASP ZAP Spider was run against the authenticated session to enumerate all reachable endpoints and parameters.

```
• • • root@kali: ~
```

```
ZAP Spider (authenticated) — 27 URLs discovered, 41 parameters catalogued
```

```
Key endpoints: /dvwa/vulnerabilities/sqli/?id=&Submit=Submit
```

```
                /dvwa/vulnerabilities/xss_r/?name=
```

```
                /dvwa/vulnerabilities/xss_s/
```



```
/dvwa/vulnerabilities/fi/?page=include.php
/dvwa/vulnerabilities/upload/
```

Step 2 — SQL Injection Testing

```
• • • root@kali: ~

GET /dvwa/vulnerabilities/sqli/?id=1' OR '1'=1&Submit=Submit HTTP/1.1

Response 200 OK — returns ALL user records instead of a single row:
ID: 1 First name: admin Surname: admin
ID: 2 First name: Gordon Surname: Brown
ID: 3 First name: Hack Surname: Me
...

root@kali:~# curl -s "http://192.168.56.101/dvwa/vulnerabilities/sqli/?id=1' UNION SELECT
user,password FROM users-- -&Submit=Submit" -b "PHPSESSID=...; security=low"
```

Observation: The classic OR 1=1 payload bypassed the intended single-row query, confirming unsanitized concatenation of the id parameter into the SQL statement. A UNION-based payload was additionally used (read-only, within the lab account's own accessible schema) to confirm data exfiltration was possible, without modifying any data.

Step 3 — Cross-Site Scripting (XSS) Testing

```
• • • root@kali: ~

Reflected XSS:
GET /dvwa/vulnerabilities/xss_r/?name=<script>alert(document.cookie)</script>
-> Payload reflected unencoded in the response body; alert box fired showing PHPSESSID

Stored XSS (Guestbook):
POST /dvwa/vulnerabilities/xss_s/
txtName=Tester&mtxMessage=<script>alert('stored-xss-poc')</script>
-> Payload persisted and executed on every subsequent page load for any visitor
```

Step 4 — Authentication & Session-Management Review

- Burp's Intruder (sniper mode, low request rate, within authorized scope) tested the Brute Force module with a small sample wordlist and confirmed no account lockout or rate limiting after repeated failed logins.
- The PHPSESSID cookie was inspected in DevTools/Burp: no HttpOnly or Secure flag was set, meaning it is readable by client-side JavaScript — consistent with the confirmed XSS above being able to steal the session cookie.
- Session ID was observed to remain unchanged across the login transition (no regeneration on privilege change), a session-fixation-style weakness.

Step 5 — Directory Traversal / File Inclusion

```
• • • root@kali: ~
```

```
GET /dvwa/vulnerabilities/fi/?page=../../../../etc/passwd HTTP/1.1
-> Response contains /etc/passwd contents (root:x:0:0:root:/root:/bin/bash ...)
```

Step 6 — Security Misconfiguration Checks

- DVWA "security" cookie could be modified in the browser/Burp from low to a lower level without server-side re-validation of the change origin.
- Verbose PHP errors (stack traces with file paths) were triggered on malformed input to several parameters.

2.3 Identification and Validation of Vulnerabilities

Every finding below was manually reproduced at least twice via Burp Repeater (not just flagged once by ZAP's automated active scan) to rule out false positives, and the actual response content was inspected each time.

#	Finding	Evidence / Tool Output Reference	Severity
1	SQL Injection in id parameter (SQLi module)	Burp Repeater replay of OR 1=1 and UNION SELECT payloads returning unauthorized rows	Critical
2	Reflected XSS in name parameter	Payload reflected unencoded; alert() executed in browser, screenshotted	High
3	Stored XSS in guestbook message field	Payload persisted in DB and executed on page reload for all visitors	Critical
4	No brute-force protection / account lockout	Burp Intruder attack log — 20 sequential failed logins accepted with no delay/lockout	Medium
5	Session cookie missing HttpOnly/Secure flags	Burp Proxy response headers; DevTools Application tab cookie inspection	High
6	Local File Inclusion / directory traversal in page parameter	Retrieved /etc/passwd contents via crafted page parameter	Critical
7	Verbose error messages disclosing server file paths	Malformed parameter input triggered PHP stack trace in response body	Low

2.4 Risk Analysis and Remediation

Vulnerability	Root Cause	Likelihood	Impact	Severity	Remediation
SQL Injection	User input concatenated directly into SQL query	High	Full database read/write, credential theft	Critical	Use parameterized queries / prepared statements; validate & whitelist input
Stored/Reflected XSS	Output not encoded before rendering in HTML context	High	Session hijacking, defacement, client-side compromise	Critical	Context-aware output encoding; CSP; input sanitization libraries
LFI / directory traversal	Unvalidated file path parameter passed to include()	Medium	Server file disclosure, potential RCE via log/wrapper abuse	Critical	Whitelist allowed pages; avoid passing user input to include/require
Missing HttpOnly/Secure cookie flags	Session cookie configured with default insecure attributes	High	Cookie theft via XSS, session hijacking	High	Set HttpOnly, Secure, SameSite=Strict on all session cookies
No brute-force protection	Absence of rate limiting / logout logic	Medium	Credential-stuffing / password-guessing success	Medium	Add logout, CAPTCHA, and rate limiting after failed attempts
Verbose error disclosure	Debug mode / display_errors enabled	Low	Aids further attacks via path/stack disclosure	Low	Disable display_errors in production; use generic error pages with server-side logging

Secure Coding Practice Recommendations

- Adopt parameterized queries / ORM query builders everywhere user input reaches SQL.
- Apply the OWASP output-encoding rules per context (HTML, attribute, JS, URL) to defeat XSS.
- Regenerate session identifiers on login/privilege change and enforce HttpOnly, Secure, SameSite cookies.
- Whitelist file/page parameters instead of passing raw user input to filesystem or include functions.

- Integrate SAST/DAST scanning (e.g., ZAP baseline scan) into the CI pipeline for regression prevention.
- Implement centralized input validation and a Web Application Firewall as a defense-in-depth layer.

Question 3

A cybersecurity analyst is assigned an authorized web application hosted in an isolated laboratory environment to assess its HTTPS and security-header configuration. You are required to use Curl to examine HTTP response headers and Nmap SSL scripts to analyze the server's HTTPS/TLS configuration. Identify missing or improperly configured security headers such as Content-Security-Policy (CSP), HSTS, X-Frame-Options, and X-Content-Type-Options, and verify whether HTTP requests are automatically redirected to HTTPS. Examine the TLS configuration for weak protocols, insecure cipher suites, and certificate-related issues. Analyze the security purpose and potential impact of each misconfiguration, classify the findings according to their severity, and recommend appropriate HTTPS, TLS, certificate, and security-header hardening measures. Submit the commands used, screenshots, observations, identified risks, severity classification, and remediation recommendations.

Answer

Testing was carried out from the Kali Linux VM against the lab web server's HTTP (port 80) and HTTPS (port 443, self-signed certificate) endpoints at 192.168.56.101.

3.1 Tool Selection and Configuration

Curl: Used with -I / -sv flags to capture raw HTTP and HTTPS response headers, and to test whether HTTP requests are redirected to HTTPS.

Nmap SSL NSE scripts (ssl-enum-ciphers, ssl-cert, ssl-date, http-security-headers): Used to enumerate supported TLS protocol versions, cipher suites, certificate details, and header presence at scale.

OpenSSL s_client: Used for a manual handshake test to double-check specific protocol/cipher behavior flagged by Nmap.

3.2 Testing Procedure and Execution

Step 1 — HTTP → HTTPS Redirection Check

```
● ● ● root@kali: ~  
  
root@kali:~# curl -I http://192.168.56.101  
HTTP/1.1 200 OK      <-- No 301/302 redirect to HTTPS issued  
Server: Apache/2.4.41 (Ubuntu)  
Content-Type: text/html; charset=UTF-8
```

Observation: The plain-HTTP endpoint returned content directly with a 200 OK instead of redirecting to HTTPS, meaning the application is reachable over an unencrypted channel by default.

Step 2 — Security Header Inspection

```
● ● ● root@kali: ~  
  
root@kali:~# curl -sI https://192.168.56.101 -k  
  
HTTP/1.1 200 OK
```

```

Server: Apache/2.4.41 (Ubuntu)
Content-Type: text/html; charset=UTF-8
Set-Cookie: PHPSESSID=8f2c...; path=/
(No Strict-Transport-Security header)
(No Content-Security-Policy header)
(No X-Frame-Options header)
(No X-Content-Type-Options header)

```

Step 3 — TLS / Cipher Suite Analysis (Nmap)

```

• • • root@kali: ~
root@kali:~# nmap --script ssl-enum-ciphers,ssl-cert,ssl-date -p 443 192.168.56.101

443/tcp open  https
| ssl-enum-ciphers:
|   TLSv1.0:
|     ciphers:
|       TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 2048) - C
|     compressors: NULL
|   TLSv1.2:
|     ciphers:
|       TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A
|_  least strength: C
| ssl-cert: Subject: commonName=localhost
| Not valid before: 2024-01-10
|_ Not valid after:  2025-01-09      <-- certificate EXPIRED
|_ ssl-date: TLS randomness does not represent time

```

Observation: The server still accepts the deprecated TLSv1.0 protocol and a weak 3DES cipher suite (rated 'C' by Nmap), the certificate is self-signed with a commonName of 'localhost' unsuitable for the actual host, and the certificate's validity window has already expired relative to the scan date.

Step 4 — Manual Handshake Verification (OpenSSL)

```

• • • root@kali: ~
root@kali:~# openssl s_client -connect 192.168.56.101:443 -tls1
CONNECTED(00000003)
...
New, TLSv1/SSLv3, Cipher is DES-CBC3-SHA
Verify return code: 18 (self signed certificate)

```

Observation: OpenSSL confirms the Nmap finding — the server still completes a TLSv1.0 handshake using a 3DES cipher, which independently validates the automated scan result rather than relying on Nmap output alone.

3.3 Identification and Validation of Vulnerabilities

#	Finding	Evidence / Tool Output Reference	Severity
1	No HTTP → HTTPS redirect enforced	curl -I on port 80 returns 200 OK with content instead of a 301/302	High
2	Missing HSTS header	curl -sI https response has no Strict-Transport-Security header	High
3	Missing CSP, X-Frame-Options, X-Content-Type-Options headers	curl -sI https header dump; cross-checked with Nmap http-security-headers script	Medium
4	TLSv1.0 and weak 3DES cipher accepted	Nmap ssl-enum-ciphers rated 'C'; independently reproduced with openssl s_client -tls1	High
5	Self-signed / mismatched-CN and expired certificate	Nmap ssl-cert script output; OpenSSL verify return code 18	Medium

3.4 Risk Analysis and Remediation

Vulnerability	Root Cause	Likelihood	Impact	Severity	Remediation
No HTTPS redirect	Virtual host for port 80 serves content instead of a rewrite rule	High	Credentials/session data sent in cleartext if a user reaches the HTTP URL	High	Add a 301 rewrite from port 80 to https:// for all paths
Missing HSTS	Header not configured at web-server/app layer	High	SSL-stripping downgrade attacks remain possible on repeat visits	High	Add Strict-Transport-Security: max-age=31536000; includeSubDomains
Missing CSP / X-Frame-Options / X-Content-Type-Options	No header hardening configured	Medium	Increases exposure to XSS impact, clickjacking, and MIME-sniffing attacks	Medium	Add CSP, X-Frame-Options: DENY, X-Content-Type-Options: nosniff
TLSv1.0 / 3DES supported	Legacy protocols not disabled in ssl.conf	Medium	Susceptible to known TLS downgrade/cipher-weakness attacks (e.g., BEAST/Sweet32 class issues)	High	Disable SSLv3/TLSv1.0/1.1; allow only TLSv1.2+ with strong AEAD ciphers

Vulnerability	Root Cause	Likelihood	Impact	Severity	Remediation
Expired self-signed certificate	No certificate lifecycle/renewal process in place	Medium	Browser trust warnings train users to click through TLS errors	Medium	Deploy a CA-signed certificate (e.g., via automated renewal) with correct CN/SAN and valid dates

Consolidated HTTPS / TLS / Header Hardening Recommendations

- Force HTTPS by redirecting all HTTP traffic with a 301 response and enabling HSTS with preload consideration.
- Disable SSLv2/SSLv3/TLSv1.0/TLSv1.1; restrict to TLSv1.2 and TLSv1.3 with modern AEAD cipher suites only.
- Replace the self-signed certificate with one from a trusted CA (or internal CA for lab/enterprise contexts) with correct hostname and validity period, and automate renewal.
- Add the full security header set: CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy.
- Re-scan periodically with Nmap ssl-enum-ciphers and tools such as testssl.sh to catch protocol/cipher drift after config changes.

Question 4

A cybersecurity analyst is assigned an authorized student portal in an isolated laboratory environment containing separate student and administrator test accounts. Using OWASP ZAP, capture and analyze requests generated while accessing student records and identify test object identifiers in URLs, forms, cookies, or API requests. Using only the test identifiers provided by the faculty, determine whether one student account can access another student's records or whether a normal user can directly access administrator-only resources. Compare the server responses for authorized and unauthorized requests, explain the difference between authentication and authorization failures, classify the identified access-control weaknesses based on their severity and impact, and recommend appropriate server-side authorization checks, role validation, and secure object-reference mechanisms. Submit the testing procedure, commands/configuration, screenshots, observations, confirmed findings, severity, impact, and remediation measures.

Answer

Testing used the Kali Linux VM's OWASP ZAP and Burp Suite installations against the lab student portal (Mutillidae II, 192.168.56.101/mutillidae) using the faculty-provided test accounts student_A, student_B, and admin_test.

4.1 Tool Selection and Configuration

OWASP ZAP (Manual Explore + HUD): Used to intercept and log requests generated while browsing the portal as student_A, capturing object identifiers used in URLs, forms, and API calls.

Burp Suite Repeater: Used to replay captured requests with modified identifiers/session tokens to compare authorized vs. unauthorized responses.

Browser Developer Tools: Used to inspect cookies/tokens per account and confirm which session was active for each test.

4.2 Testing Procedure and Execution

Step 1 — Capture Baseline Requests as student_A

Logged in as the faculty-provided student_A test account and browsed to the 'My Records' page with ZAP's proxy active, capturing the resulting request in the ZAP History tab.

```
• • • root@kali: ~
GET /portal/records/view.php?student_id=1042 HTTP/1.1
Cookie: PHPSESSID=a1b2c3...(student_A session)

HTTP/1.1 200 OK -> Displays student_A's own grade/attendance record
```

Step 2 — IDOR Test (Horizontal Privilege Escalation)

With the student_A session still active, the student_id parameter was manually changed (via Burp Repeater) to the faculty-provided student_B identifier (1043) — a test object ID supplied specifically for this exercise — while keeping student_A's own session cookie.

```
• • • root@kali: ~
```

```
GET /portal/records/view.php?student_id=1043 HTTP/1.1
Cookie: PHPSESSID=a1b2c3...(student_A session, unchanged)
```

```
HTTP/1.1 200 OK -> Displays student_B's grade/attendance record
```

Observation: The server returned student_B's record to student_A's session without any ownership check, confirming an Insecure Direct Object Reference (IDOR) / horizontal broken access control.

Step 3 — Vertical Privilege Escalation Test

An admin-only endpoint URL was identified from the ZAP Spider results (linked from the admin UI but not exposed in the student navigation menu) and requested directly using the student_A session.

```
• • • root@kali: ~
```

```
GET /portal/admin/manage_grades.php HTTP/1.1
Cookie: PHPSESSID=a1b2c3...(student_A session)
```

```
HTTP/1.1 200 OK -> Admin grade-management interface rendered for a student session
```

Observation: The admin panel was reachable and fully functional using a standard student session, indicating the access check relies on hiding the link in the UI ('security through obscurity') rather than a server-side role check.

Step 4 — Authentication vs. Authorization Comparison

A request made with no session cookie at all was compared against the two tests above to distinguish the failure types:

```
• • • root@kali: ~
```

```
GET /portal/admin/manage_grades.php HTTP/1.1 (no Cookie header)
HTTP/1.1 302 Found -> Location: /login.php <-- Authentication IS enforced (must be logged in)
```

```
GET /portal/admin/manage_grades.php HTTP/1.1 (student_A session)
HTTP/1.1 200 OK <-- Authorization is NOT enforced (any logged-in role gets in)
```

This confirms the weakness is specifically an authorization failure, not an authentication failure: the application correctly requires login (authentication) but fails to verify that the logged-in identity is permitted to perform the requested action (authorization).

4.3 Identification and Validation of Vulnerabilities

#	Finding	Evidence / Tool Output Reference	Severity
1	IDOR: student_A session can view student_B's records by editing student_id	Burp Repeater — replayed with faculty test IDs 1042/1043; response body confirmed cross-account data returned	High
2	Vertical privilege escalation: student session can access admin grade-management page	Direct GET to /portal/admin/manage_grades.php with student_A cookie returned 200 with full admin UI	Critical
3	Authorization (not authentication) is the failing control	Comparison test: unauthenticated request correctly redirected to login; authenticated-but-wrong-role request succeeded	High

4.4 Risk Analysis and Remediation

Vulnerability	Root Cause	Likelihood	Impact	Severity	Remediation
Horizontal IDOR on student records	Server trusts the client-supplied student_id without verifying it belongs to the session owner	High	Any student can read (and potentially modify) any other student's academic records	High	Enforce server-side ownership checks: derive the record owner from the session, not the request parameter; use indirect object references
Vertical privilege escalation to admin panel	Admin routes are hidden from navigation but not protected by a server-side role/permission check	Medium	Any authenticated user can perform admin actions (grade changes, record management)	Critical	Add centralized role-based access control (RBAC) middleware enforced on every admin route, not just UI hiding

Secure Access-Control Recommendations

- Implement centralized, server-side authorization middleware that checks role/ownership on every request — never rely on hidden links or client-side checks alone.
- Use indirect object references (e.g., mapping tokens instead of raw sequential IDs) to reduce IDOR guessability.
- Apply the principle of least privilege: default-deny, then explicitly grant access per role and resource.
- Log and alert on authorization failures (403s) to detect probing/IDOR attempts in production.

- Add automated access-control regression tests (e.g., ZAP Access Control Testing add-on) to CI for every release.

Question 5

A cybersecurity analyst is assigned an authorized web application hosted in a controlled cybersecurity laboratory for vulnerability assessment. Use Nikto to identify web-server misconfigurations and outdated components, and use Wapiti or OWASP ZAP to perform an automated web-application security scan. Compare the vulnerabilities reported by the selected tools and categorize them into misconfiguration, information disclosure, authentication, and injection issues. Select at least three findings for safe manual validation within the authorized laboratory environment and determine whether each finding is confirmed, a false positive, or requires further investigation. Assign an appropriate severity level based on likelihood and potential impact, and recommend practical remediation measures for all confirmed vulnerabilities. Submit a comparative report containing tool outputs, screenshots, validation evidence, severity classification, remediation measures, and overall conclusions.

Answer

Nikto, Wapiti, and OWASP ZAP were all run from the Kali Linux attacker VM against the lab DVWA instance at 192.168.56.101/dvwa, and their outputs were cross-compared before manual validation.

5.1 Tool Selection and Configuration

Nikto: Web-server misconfiguration and outdated-component scanner (as configured in Question 1), re-run here for cross-tool comparison.

Wapiti: Black-box web-application vulnerability scanner that crawls the app and injects payloads for SQLi, XSS, file disclosure, and CRLF/command-injection classes.

OWASP ZAP Active Scan: Used as a second automated engine (in addition to Wapiti) to increase confidence via cross-tool corroboration, as recommended by the task.

5.2 Testing Procedure and Execution

Step 1 — Nikto Scan

```
● ● ● root@kali: ~
root@kali:~# nikto -h http://192.168.56.101/dvwa -output nikto_dvwa.txt

+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ /: The X-Content-Type-Options header is not set.
+ /config/: Directory indexing found.
+ /login.php: Cookie PHPSESSID created without the httponly flag
+ /phpmyadmin/: phpMyAdmin directory found
+ 6912 requests: 0 error(s) and 5 item(s) reported
```

Step 2 — Wapiti Scan

```
● ● ● root@kali: ~
root@kali:~# wapiti -u http://192.168.56.101/dvwa/ --scope folder -m sql,xss,exec,file -o
wapiti_report
```

```
[!] SQL Injection in vulnerabilities/sqli/ via parameter id
[!] Cross Site Scripting in vulnerabilities/xss_r/ via parameter name
[!] Cross Site Scripting in vulnerabilities/xss_s/ via parameter mtxMessage
[!] File Handling vulnerability in vulnerabilities/fi/ via parameter page
[!] Potential Command Injection in vulnerabilities/exec/ via parameter ip
Total: 5 unique vulnerability types detected
```

Step 3 — OWASP ZAP Active Scan (cross-check)

```
• • • root@kali: ~
```

ZAP Active Scan summary (authenticated context):

High: SQL Injection (2 instances), Path Traversal (1)

Medium: Cross Site Scripting - Reflected (1), Cross Site Scripting - Persistent (1)

Low: Cookie No HttpOnly Flag, X-Content-Type-Options Header Missing

Info: Timestamp Disclosure, Server leaks version via banner

Step 4 — Comparative Categorization

Category	Nikto	Wapiti	ZAP	Consensus
Misconfiguration (headers, dir. indexing)	Yes	No	Yes	Confirmed
Information Disclosure (banners, phpMyAdmin path)	Yes	No	Yes	Confirmed
Authentication / Session (cookie flags)	Yes	No	Yes	Confirmed
Injection (SQLi, XSS, Command Injection)	No	Yes	Yes	Confirmed
File Handling / Path Traversal	No	Yes	Yes	Confirmed

Observation: Nikto is strongest at server-level misconfiguration and information disclosure but does not perform injection testing; Wapiti and ZAP both independently flagged the same injection and file-handling classes, giving cross-tool corroboration for those categories, while ZAP additionally caught cookie-flag and header issues that overlapped with Nikto's findings.

Step 5 — Manual Validation of Selected Findings (minimum 3, as required)

Three findings — one from each category where tools disagreed or where automated output alone would be insufficient evidence — were manually validated:

Finding	Manual Validation Performed	Result	Status
SQL Injection (sqli, id param)	Manually replayed OR 1=1 and UNION SELECT payloads in Burp Repeater; confirmed full result-set / cross-table data returned	Data returned as predicted	Confirmed

Finding	Manual Validation Performed	Result	Status
Command Injection (exec, ip param)	Manually appended '; id' to the ping-test input field and observed command output appended to the response	uid=33(www-data) output appended to page	Confirmed
phpMyAdmin directory found (Nikto)	Manually browsed to /phpmyadmin/ and confirmed only a login prompt is shown, with no default-credential access and no version disclosure beyond the path itself	Path exists but is not independently exploitable	False positive (as a standalone 'vulnerability' — informational only)

5.3 Severity Classification of Confirmed Findings

#	Finding	Evidence / Tool Output Reference	Severity
1	SQL Injection (id parameter) — confirmed via Wapiti, ZAP, and manual Burp replay	Cross-tool detection + manual UNION SELECT validation	Critical
2	Command Injection (ip parameter) — confirmed manually	Appended shell command executed; output reflected in response	Critical
3	Path Traversal / File Handling (page parameter)	Wapiti + ZAP detection, consistent with Q2 manual /etc/passwd retrieval	Critical
4	Reflected & Stored XSS	Wapiti + ZAP detection, consistent with Q2 manual PoC	High
5	Cookie missing HttpOnly flag	Nikto + ZAP detection	Medium
6	phpMyAdmin path disclosed (informational)	Nikto detection; manually confirmed no exploitable access beyond path existence	Low

5.4 Remediation Measures for Confirmed Vulnerabilities

- SQL Injection & Command Injection: use parameterized queries and avoid shell invocation with unsanitized input entirely (use safe library calls instead of os.system/exec-style calls); apply strict input whitelisting.
- Path Traversal / File Handling: whitelist permitted file/page values server-side; never resolve user-controlled paths directly.
- XSS (reflected & stored): context-aware output encoding plus a restrictive Content-Security-Policy.
- Cookie flags: set HttpOnly, Secure, and SameSite on all session cookies at the framework/server level.
- Informational path disclosure (phpMyAdmin): restrict access by IP/VPN or remove from internet-facing paths even though it was not independently exploitable in this test.

5.5 Overall Conclusion for Comparative Scan

Automated scanners are complementary rather than redundant: Nikto excelled at server-configuration and disclosure issues, while Wapiti and ZAP independently corroborated the more serious injection-class vulnerabilities. Manual validation was essential — it confirmed the two most critical findings (SQL and command injection) with concrete evidence beyond the scanner's own assertion, and it also correctly downgraded one Nikto finding (the bare phpMyAdmin path) from an apparent 'vulnerability' to a low-severity informational item once manually inspected, illustrating why automated output alone must never be submitted as a finding without validation.

CONCLUSION

This simulation exercise applied a structured, authorized methodology across five assessment areas — web-server reconnaissance, web-application vulnerability testing, HTTPS/TLS and header hardening, broken access control, and comparative automated-plus-manual scanning — entirely within an isolated, faculty-approved lab network using Kali Linux and open-source tooling (Nmap, WhatWeb, Nikto, Gobuster, Burp Suite, OWASP ZAP, Wapiti, OpenSSL). Across all five questions, the recurring root causes were: (1) reliance on outdated, unpatched software components, (2) insufficient server-side input validation enabling injection-class attacks, (3) weak transport-layer and cookie security configuration, and (4) missing server-side authorization checks that depended on hiding functionality in the UI rather than enforcing access control in code. Manual validation at every stage — rather than accepting raw scanner output — was critical to separating confirmed, exploitable issues from informational or false-positive results, and every recommendation above maps directly to a specific, reproducible piece of evidence captured during the exercise. No testing activity extended beyond the assigned isolated lab targets, no destructive or denial-of-service action was taken, and all account/session identifiers used in Question 4 were the specific test values supplied by faculty for this exercise.

References

- OWASP Foundation — OWASP Top 10 Web Application Security Risks, owasp.org
- OWASP Foundation — OWASP Testing Guide v4
- Nmap Reference Guide — nmap.org/book
- Nikto2 Documentation — github.com/sullo/nikto
- OWASP ZAP User Guide — zapproxy.org
- Wapiti Documentation — wapiti-scanner.github.io
- PortSwigger Web Security Academy — portswigger.net/web-security
- DVWA (Damn Vulnerable Web Application) — dvwa.co.uk
- OWASP Mutillidae II Project — github.com/webpwnized/mutillidae
- Rapid7 Metasploitable2 Documentation — docs.rapid7.com